Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales

Simulación y análisis de congestión en redes bajo distintos esquemas de encaminamiento

Monografía Final

Cátedra: Redes de Computadoras

Docentes: Santiago Martín Henn, Facundo Nicolás Oliva Cuneo

Grupo NetRunners

Bastida, Lucas Ramiro
Contessi, Rubén Andrés
Garay, Ignacio Hernán
Giraudo, Juan Pablo
Peñaloza, Gonzalo Adrián
Quinteros del Castillo, Tomás Agustín
Vasconsellos Blason, Benjamín Alejandro

Introducción

En una red de datos el tráfico nunca es parejo: depende de cuántos usuarios hay conectados en ese momento, qué aplicaciones están usando y hasta de la hora del día. Cuando ese tráfico crece demasiado y se acerca al límite que pueden manejar los enlaces o los routers, aparece la congestión. Ahí empiezan los problemas: paquetes que tardan más de lo normal, pérdidas y descartes en los dispositivos intermedios.

Este trabajo estudia ese problema de forma práctica mediante la simulación de una red y el análisis de su comportamiento bajo distintas condiciones de tráfico. A partir de las observaciones realizadas, buscamos identificar cómo varían el rendimiento y la estabilidad de la comunicación a medida que aumentan las exigencias sobre la red.

Nos interesan puntualmente dos cosas. Por un lado, el papel del encaminamiento en la disponibilidad de la comunicación ante cambios en la red, comparando rutas estáticas con los protocolos dinámicos RIPv2 y OSPF. Por otro, cómo influye el tipo de tráfico circulante, ya que no responde igual TCP, que regula su envío al detectar congestión, que un flujo UDP en tiempo real, que puede seguir transmitiendo aunque la red esté sobrecargada.

A lo largo del trabajo se compararán los resultados obtenidos con distintos esquemas de encaminamiento y tipos de tráfico, relacionándolos con los mecanismos de control de congestión estudiados en la materia.

Objetivos

Objetivo general

Analizar de forma experimental el comportamiento de una red frente a condiciones de congestión creciente y fallas de enlace, evaluando la influencia del encaminamiento y del tipo de tráfico en la disponibilidad y el rendimiento de la comunicación.

Objetivos específicos

- Configurar una topología de red que permita comparar diferentes esquemas de encaminamiento, manteniendo constantes su estructura y las condiciones de carga.
- Generar tráfico creciente entre los extremos de la red hasta alcanzar condiciones de congestión controlada.
- Registrar y analizar indicadores de congestión, tales como pérdida de paquetes, latencia, retransmisiones y variaciones en el rendimiento.
- Comparar la respuesta de flujos de tráfico con mecanismos de control de congestión diferentes frente a una misma condición de carga.
- Relacionar los resultados observados con los mecanismos de control de congestión y recuperación de la comunicación.
- Medir y comparar la capacidad de recuperación de la red ante la caída de un enlace según el esquema de encaminamiento utilizado.
- Comparar los resultados de los ensayos según pérdida de paquetes, latencia, estabilidad del tráfico y tiempo de recuperación.

1 Herramientas y entorno

Antes de meternos con la parte experimental nos parece importante dejar documentado con qué herramientas vamos a trabajar y por qué.

Si bien Packet Tracer resulta adecuado para las prácticas propuestas en la materia, para este trabajo vamos a armar las topologías en GNS3 (`https://www.gns3.com/`). La diferencia clave es que GNS3 corre cada router y switch como una máquina virtual real, con interfaces de red que sí existen a nivel del sistema operativo del host; Packet Tracer, en cambio, es un simulador cerrado donde el tráfico nunca sale de su propio motor interno. Su modo Simulación permite analizar el tráfico dentro de la plataforma, pero queda restringido a ese entorno: no es el tráfico real de una placa de red. Como para nosotros es central poder capturar con Wireshark, GNS3 es la opción que nos permite hacerlo sin ningún truco.

Para los routers vamos a usar la imagen de MikroTik CHR (*Cloud Hosted Router*), disponible en `https://mikrotik.com/download`, que corre RouterOS, el mismo sistema operativo de los routers MikroTik físicos. La elegimos por dos motivos: primero, porque MikroTik es una marca económica y muy extendida en redes de pequeña escala en Argentina, así que las configuraciones que probemos acá pueden servir de referencia para una implementación sobre equipos reales (retomamos esto en la Sección 4); segundo, porque su licencia gratuita permite utilizar las funciones de encaminamiento necesarias para los ensayos. Si bien esta licencia limita la velocidad de las interfaces a 1 Mbps, las condiciones de tráfico de la simulación se establecerán por debajo de ese valor. De esta manera, la limitación no condicionará la comparación entre los esquemas de encaminamiento ni el análisis de la congestión, sino que definirá un marco de capacidad conocido para los experimentos.

Para el análisis del tráfico vamos a utilizar Wireshark (`https://www.wireshark.org/`) y TShark (`https://www.wireshark.org/docs/man-pages/tshark.html`), herramientas también empleadas en las actividades de la cátedra. Wireshark permitirá examinar de forma interactiva las capturas obtenidas en los enlaces de la topología, inspeccionar encabezados y campos de los paquetes, aplicar filtros y reconocer eventos como retransmisiones, pérdidas, variaciones de retardo o mensajes de los protocolos de encaminamiento. TShark, su versión de línea de comandos y basada en el mismo motor de análisis, permitirá procesar los archivos PCAP de manera automatizada y extraer los campos o métricas necesarios con criterios repetibles entre los distintos ensayos. El uso conjunto de ambas herramientas permitirá complementar la interpretación visual de las capturas con un análisis cuantitativo reproducible.

El tráfico de prueba se generará desde los extremos de la topología mediante máquinas Linux livianas integradas en GNS3. Para los flujos de transporte se utilizará `iperf3` (`https://github.com/esnet/iperf`), tanto en modo TCP como UDP: el primero permitirá observar el comportamiento de un protocolo que adapta su transmisión ante la congestión, mientras que el segundo permitirá enviar tráfico a una tasa definida y registrar métricas como pérdida y variación del retardo. Además, las pruebas de HTTP/1.1 se realizarán con Nginx (`https://nginx.org/`) como servidor del recurso de prueba y `wrk` (`https://github.com/wg/wrk`) como generador de carga desde el cliente. El uso de estas herramientas permitirá evaluar distintos tipos de tráfico bajo condiciones comparables; la Sección 3 detalla los parámetros y las métricas que se registrarán en cada ensayo.

Plataforma de ejecución y versiones

Los experimentos se realizarán localmente en una notebook ASUS X556UB con procesador Intel Core i7-6500U a 2,50 GHz (dos núcleos y cuatro hilos), 12 GiB de memoria RAM y virtualización Intel VT-x habilitada. El sistema anfitrión será Linux Mint 22.3 de 64 bits, con

kernel Linux 6.8.0-149-generic. La topología no utilizará una GNS3 VM ni un servidor remoto: los cinco routers y los dos endpoints se ejecutarán en el equipo anfitrión. Cada router CHR se configurará en QEMU con una vCPU, 512 MiB de RAM y cuatro adaptadores *virtio-net-pci*; los endpoints serán contenedores Docker.

Con el fin de mantener condiciones de ejecución comparables, los ensayos se realizarán sobre una carga de referencia estable del equipo anfitrión. Se evitarán cambios relevantes en su utilización durante las mediciones, de modo que las diferencias observadas puedan atribuirse al comportamiento de la red simulada y no a variaciones de recursos de la plataforma de ejecución.

La siguiente tabla registra las versiones que se emplearán.

Componente	**Versión**	**Uso**
GNS3	2.2.61	Diseño y ejecución de la topología local.
QEMU	8.2.2	Ejecución de los cinco routers virtuales.
MikroTik CHR / RouterOS	7.24.6	Rutas estáticas, RIPv2, OSPF y colas de los routers.
Docker	29.1.3	Ejecución de los nodos cliente y servidor.
Debian (endpoints)	12 (bookworm)	Sistema operativo base de los nodos cliente y servidor.
`iperf3` (endpoints)	3.12	Generación y medición de tráfico TCP y UDP.
Nginx (endpoint servidor)	1.22.1	Servicio HTTP/1.1 del recurso de prueba.
`wrk` (endpoint cliente)	4.1.0-3+b2	Carga HTTP/1.1 y medición de solicitudes.
Wireshark / TShark	4.6.9	Captura y análisis de archivos PCAP.
`tc` (endpoints)	iproute2 6.1.0	Control complementario de tráfico cuando corresponde.

Tabla A. Plataforma de ejecución y versiones del software previsto para la simulación.

2 Marco teórico

Esta sección desarrolla los tres ejes que ya habíamos adelantado en el plan de trabajo: control de congestión (que es el centro de la monografía), encaminamiento y capa de aplicación. En cada uno tratamos de apoyarnos primero en lo que da Stallings, que es la bibliografía de la cátedra, y sólo recurrimos a otras fuentes cuando el libro no cubre algo con el detalle que necesitamos, como pasa con los protocolos de encaminamiento concretos y con algunos mecanismos de congestión que quedaron desactualizados desde que se escribió.

2.1 Control de congestión

La congestión aparece cuando, en un nodo de la red (un conmutador, un router), el número de paquetes que llega empieza a acercarse al límite de lo que ese nodo puede procesar y reenviar. Cada puerto de un nodo tiene colas de entrada y de salida, y si los paquetes llegan más rápido de lo que se pueden transmitir, esas colas empiezan a crecer. Como ninguna memoria es infinita, tarde o temprano el nodo se queda sin lugar donde guardar paquetes nuevos y tiene que empezar a descartarlos (Stallings, Sección 13.1).

Lo interesante es ver qué pasa con el rendimiento de la red a medida que crece la carga. En un esquema ideal, el rendimiento crecería linealmente con la carga ofrecida hasta llegar a la capacidad máxima de la red, y a partir de ahí se mantendría constante. En la práctica no pasa eso: como las memorias de los nodos son finitas, llega un punto en el que la red entra en una congestión moderada (el rendimiento sigue creciendo pero cada vez más despacio) y después en una congestión severa, donde el rendimiento real empieza a caer aunque la carga siga aumentando. Esto pasa principalmente porque, cuando las colas se llenan, los nodos empiezan a descartar paquetes, los orígenes los retransmiten, y esa retransmisión suma todavía más carga a una red que ya estaba saturada (Stallings, Sección 13.1). El Gráfico 1 resume esta idea comparando el caso ideal con el real.

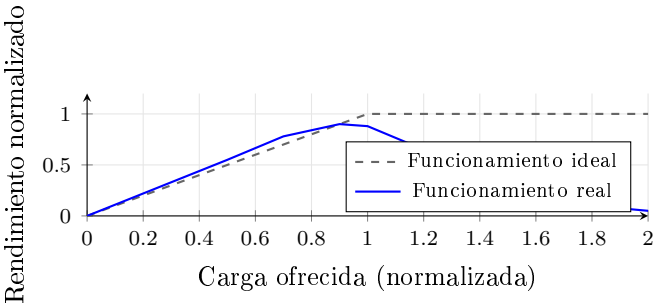


Gráfico 1. Rendimiento ideal vs. real de una red frente al aumento de la carga ofrecida (elaboración propia en base a Stallings, Figuras 13.3 y 13.4).

Para evitar llegar a ese escenario, Stallings describe varios mecanismos de control de congestión, cada uno pensado para intervenir en un punto distinto de la red. La contrapresión actúa a nivel de enlace: el nodo congestionado frena a sus vecinos directos, y esa restricción se va propagando hacia atrás hasta llegar al origen. Los paquetes de obstrucción son más directos: el nodo congestionado le manda al origen un paquete de control (el caso típico que da Stallings es el *Source Quench* de ICMP) pidiéndole que baje la velocidad de envío. Después están las dos formas de señalización: la implícita, que es la que usa TCP, donde el emisor infiere que hay congestión por el aumento del retardo o por la pérdida de paquetes sin que la red le avise nada explícitamente; y la explícita, donde la red marca bits en los paquetes (hacia atrás o hacia adelante, como los bits BECN/FECN en retransmisión de tramas) para notificarles a los extremos que hay congestión en el camino (Stallings, Sección 13.2).

Vale la pena una aclaración sobre estos dos últimos mecanismos, porque quedaron un poco desactualizados desde que Stallings los describió. El *Source Quench* de ICMP está formalmente deprecado desde el RFC 6633 (2013): en la práctica, casi ningún sistema operativo moderno lo genera ni lo procesa, así que si lo buscamos en nuestras capturas probablemente no lo vamos a encontrar. La señalización explícita tampoco se quedó en BECN/FECN: su equivalente moderno en redes IP es ECN (*Explicit Congestion Notification*, RFC 3168), que usa dos bits del encabezado IP para que un router marque un paquete en lugar de descartarlo cuando una cola empieza a llenarse, y el receptor le devuelve ese aviso al emisor para que reduzca la ventana sin necesidad de perder el paquete. Para que ECN tenga margen donde marcar en vez de descartar, además, conviene que los routers usen gestión activa de colas, como RED (*Random Early Detection*), en lugar del descarte de cola simple (*tail drop*) que asumimos por defecto: RED empieza a descartar o marcar paquetes al azar un poco antes de que la cola se llene del todo, para avisar la congestión de a poco en lugar de que se corten muchos flujos TCP al mismo tiempo.

Además de estos mecanismos, Stallings menciona la gestión de tráfico como un conjunto de criterios adicionales para administrar la congestión: la imparcialidad (que todos los flujos sufran la congestión en una medida parecida, en vez de que un nodo descarte siempre los paquetes del mismo origen), la calidad de servicio (dar prioridad a los flujos más sensibles al retardo, como voz o video, por sobre los que no lo son) y las reservas (acordar de antemano una velocidad de transmisión, como hacen las redes ATM) (Stallings, Sección 13.3). Esto nos va a servir más adelante para justificar por qué comparamos tráfico HTTP contra un flujo UDP en tiempo real: son dos tipos de tráfico con requisitos bien distintos frente a la congestión (Stallings, Sección 13.3).

La reacción específica de TCP

Como una parte importante de nuestro tráfico simulado va a ser HTTP, que corre sobre TCP, nos interesa particularmente cómo reacciona TCP ante la congestión. TCP no recibe ninguna señal explícita de la red: infiere que hay congestión cuando detecta que un segmento se perdió (porque expiró un temporizador de retransmisión) o porque el retardo de ida y vuelta (RTT) empieza a crecer (Stallings, Sección 20.3).

Para controlar cuánto puede enviar en cada momento, TCP usa una ventana de congestión (*cwnd*), que se suma al mecanismo de crédito que ya conocíamos de control de flujo. Cuando se abre una conexión nueva, *cwnd* arranca en un segmento y se duplica en cada tiempo de ida y vuelta mientras lleguen confirmaciones: esto es lo que se llama arranque lento, y hace que la ventana crezca exponencialmente al principio. Cuando *cwnd* llega a un umbral (*ssthresh*), TCP pasa a una fase más conservadora, el ajuste dinámico o evitación de congestión, donde la ventana crece de a un segmento por cada tiempo de ida y vuelta en lugar de duplicarse. Si en algún momento se pierde un segmento, TCP reduce el umbral a la mitad de la ventana actual, vuelve a poner *cwnd* en uno y arranca todo el proceso de nuevo (Stallings, Sección 20.3). El Gráfico 2 ilustra cómo se ve esta evolución a lo largo del tiempo.

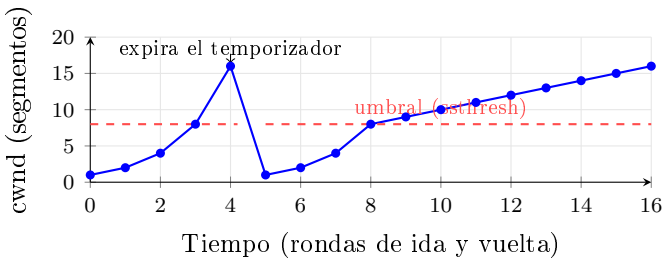


Gráfico 2. Evolución típica de la ventana de congestión de TCP: arranque lento (crecimiento exponencial), ajuste dinámico (crecimiento lineal) y caída ante una pérdida (elaboración propia en base a Stallings, Figura 20.13).

Lo que describimos recién es el comportamiento ante un *timeout*: TCP asume lo peor y vuelve a arrancar desde *cwnd* en uno. Pero hay otra forma de detectar la pérdida que no necesita esperar a que venza el temporizador: si el receptor sigue recibiendo segmentos fuera de orden, manda el mismo ACK repetido (ACK duplicado) por cada uno que llega; cuando el emisor ve tres ACKs duplicados seguidos, interpreta que un segmento se perdió sin que haga falta esperar el *timeout*, y lo retransmite de inmediato – esto es la retransmisión rápida. Como en este caso la red no necesariamente está tan congestionada como en un *timeout* (si siguen llegando segmentos postériores, la red todavía está entregando paquetes), TCP no vuelve a arrancar de cero: reduce el umbral a la mitad y ajusta *cwnd* a ese nuevo umbral en lugar de volver a uno, lo que se conoce como recuperación rápida. Esta diferencia es justamente uno de los indicadores que vamos a buscar en las capturas: una caída de *cwnd* por ACKs duplicados debería verse distinta – menos abrupta – que una caída por *timeout*.

Para decidir cuánto esperar antes de retransmitir un segmento, TCP necesita estimar el RTT. El algoritmo de Jacobson suaviza esa estimación combinando un promedio exponencial del RTT con una estimación de su variabilidad, en lugar de usar sólo un promedio simple, lo que lo hace mucho más sensible a cambios bruscos en las condiciones de la red (Stallings, Sección 20.3). El algoritmo de Karn, por su parte, resuelve un problema puntual: cuando un segmento se retransmite, no se puede saber con certeza si la confirmación que llegó corresponde al envío original o a la retransmisión, así que esa muestra de RTT simplemente se descarta para no distorsionar la estimación (Stallings, Sección 20.3). Estos dos mecanismos son los que vamos a buscar reflejados en las capturas de Wireshark cuando analicemos cómo reacciona TCP durante la simulación.

Una aclaración importante antes de pasar a las capturas: todo lo que describimos en esta sección (arranque lento, ajuste dinámico, caída a la mitad del umbral) corresponde a los algoritmos clásicos que desarrolla Stallings, básicamente Tahoe y Reno. El Linux que vamos a usar en las puntas de GNS3, sin embargo, usa CUBIC como algoritmo de control de congestión por defecto desde hace ya varios años, que ajusta la ventana con una función cúbica del tiempo transcurrido desde la última pérdida en lugar del crecimiento lineal de Reno, pensada para aprovechar mejor enlaces de alta velocidad y alta latencia. Lo anticipamos acá para no sorprendernos después: las capturas reales probablemente no van a calcar la forma exacta del Gráfico 2, aunque el patrón general – crecimiento, caída ante pérdida, nuevo crecimiento – se va a mantener.

2.2 Encaminamiento

El segundo eje tiene que ver con cómo decide la red por dónde mandar cada paquete, lo cual se vuelve clave cuando parte de la red empieza a congestionarse. Stallings presenta cuatro estrategias generales de encaminamiento, resumidas en la Tabla 1.

Estrategia	Cómo decide la ruta	Ventaja principal	Desventaja principal
Estática	Ruta fija precalculada para cada par origen-destino	Simplicidad, buen funcionamiento con carga estable	No reacciona ante fallos ni ante congestión
Inundación	Cada nodo reenvía por todos los enlaces salvo por el que llegó el paquete	Máxima robustez, asegura la entrega si existe alguna ruta	Genera una cantidad de tráfico muy alta
Aleatoria	El nodo elige al azar (o según una probabilidad) un enlace de salida	Sencilla, reparte la carga sin necesitar información de la red	No sigue necesariamente la ruta de menor coste
Adaptable	Las decisiones cambian según el estado real de la red (fallos, congestión)	Mejora el rendimiento percibido y ayuda a esquivar zonas congestionadas	Más compleja y puede generar oscilaciones si reacciona mal

Tabla 1. Estrategias de encaminamiento (Stallings, Sección 12.2).

Dentro del encaminamiento adaptable, que es el que nos interesa porque es el que usan los protocolos dinámicos como RIP y OSPF, Stallings desarrolla dos algoritmos de mínimo coste que son la base de casi todo el encaminamiento moderno. El algoritmo de Dijkstra necesita conocer la topología completa de la red (todos los enlaces y sus costes) y va construyendo la ruta en etapas, incorporando en cada paso el nodo más cercano que todavía no forma parte del camino; es la base de OSPF. El de Bellman-Ford es más austero: a cada nodo le alcanza con conocer el coste a sus vecinos directos, que va intercambiando periódicamente con ellos en forma de vectores de distancia; es la base de RIP (Stallings, Sección 12.3, p. 397–401).

Acá es donde encontramos el límite de Stallings para nuestro trabajo: el libro desarrolla los algoritmos de base, pero no entra en el detalle de los protocolos RIP, OSPF, EIGRP o BGP como tales. Como vamos a simular justamente RIP y OSPF en GNS3, para esta parte puntual recurrimos a Kurose y Ross, que sí los desarrolla en profundidad y es la referencia más usada en la facultad como complemento de Stallings para esto. La Tabla 2 resume la comparación que vamos a usar como base para interpretar lo que veamos en las capturas.

	RIP	OSPF
Algoritmo base	Bellman-Ford (vector distancia)	Dijkstra (estado de enlace)
Métrica	Número de saltos (máximo 15)	Costo configurable, típicamente en función del ancho de banda
Temporizadores	Actualización 30 s, inválido 180 s, purga 240 s	Hello 10 s, Dead 40 s
Transporte	UDP puerto 520, multicast 224.0.0.9	IP directo, protocolo número 89
Convergencia	Más lenta, puede sufrir el problema de "cuenta a infinito"	Más rápida ante cambios en la red
Escala típica	Redes pequeñas o medianas	Redes grandes, organizadas en áreas

Tabla 2. RIP vs. OSPF (fuente: Kurose y Ross, *Redes de Computadoras: un Enfoque Descendente*; RFC 2453 y RFC 2328).

Para poder interpretar lo que capturemos con Wireshark, nos interesa bajar un escalón más, de los algoritmos de base a cómo funcionan en concreto los protocolos que vamos a simular.

RIPv2 (RFC 2453) manda su tabla completa de rutas cada 30 segundos por multicast (224.0.0.9) sobre UDP, puerto 520. Si un router deja de recibir actualizaciones de un vecino durante 180 segundos, marca esa ruta como inválida, y a los 240 segundos (sin que haya llegado una actualización que la corrija) la purga definitivamente de la tabla; estos tres tiempos son justamente los que explican por qué RIP tarda tanto en darse cuenta de que algo cambió. Para evitar el problema clásico de "cuenta a infinito" (dos routers pasándose entre sí una ruta caída como si todavía existiera), RIP combina horizonte dividido (no anunciarle a un vecino una ruta que se aprendió de él mismo) con envenenamiento en reversa (anunciar esa ruta de

vuelta, pero con métrica infinita, en lugar de simplemente omitirla) y *hold-down* (ignorar por un tiempo cualquier anuncio que empeore una ruta recién caída, por si es información vieja que todavía está circulando). Las actualizaciones disparadas (mandar la tabla apenas cambia algo, sin esperar los 30 segundos) ayudan a que la convergencia no dependa únicamente del temporizador periódico.

OSPF (RFC 2328) funciona distinto: en lugar de mandar toda la tabla periódicamente, cada router arma una base de datos de estado de enlace (LSDB) a partir de anuncios (LSA, de distintos tipos según qué describan) que intercambia y sincroniza con sus vecinos solo cuando algo cambia. Los vecinos se descubren y se mantienen con paquetes Hello cada 10 segundos; si un vecino no manda un Hello en 40 segundos (el temporizador *Dead*), se lo da de baja y se recalcula. En redes multiacceso (como una LAN con varios routers), OSPF elige un router designado (DR) y uno de respaldo (BDR) para que los demás sincronicen la LSDB solo con ellos y no entre todos contra todos, y para que la red no crezca indefinidamente en una sola LSDB permite dividirla en áreas, donde el área 0 (*backbone*) conecta a todas las demás. OSPF corre directamente sobre IP, con el protocolo número 89, sin pasar por UDP ni TCP.

Dentro del mecanismo de inundación, no todos los LSA describen lo mismo. Los más relevantes para interpretar una captura son los que resume la Tabla 3.

Tipo	**Nombre**	**Qué describe**
1	Router LSA	Los enlaces directos de un router y su costo; cada router lo genera únicamente para su propia área.
2	Network LSA	La red multiacceso y los routers conectados a ella; lo genera el router designado (DR) de esa red, no cada router por separado.
3	Summary LSA	Una ruta aprendida en otra área, reinundada por un router de borde de área (ABR) hacia el resto de las áreas para que sepan cómo llegar sin conocer su topología interna.
5	AS External LSA	Una ruta aprendida fuera del dominio OSPF (por ejemplo, redistribuida desde una ruta estática o desde BGP), inundada por un router de borde de sistema autónomo (ASBR).

Tabla 3. Tipos de LSA más comunes en OSPF (RFC 2328, Sección 12).

En nuestra topología de laboratorio, chica y de una sola área, el tipo 1 alcanza para explicar casi todo lo que vamos a ver en las capturas: cuando cae `R2-R4`, lo que se inunda es un Router LSA actualizado de R2 y de R4, cada uno sin ese enlace en su lista. No vamos a tener tipo 2 (no hay ninguna LAN multiacceso con más de un router OSPF en nuestro diamante, todos los enlaces son punto a punto) ni tipo 3 o 5 (toda la topología vive en una única área 0, sin rutas redistribuidas desde afuera), pero nos pareció importante dejarlos documentados porque son justamente los que aparecerían en cualquier red OSPF un poco más grande que la nuestra, y porque explican por qué OSPF escala mejor que un vector-distancia plano: puede resumir y ocultar el detalle topológico de un área entera detrás de un único Summary LSA.

La métrica de OSPF merece una aclaración aparte, porque nos puede jugar en contra si no la ajustamos: el costo de un enlace se calcula como un ancho de banda de referencia dividido el ancho de banda real del enlace, y por defecto, en equipos Cisco, esa referencia es 100 Mbps. El problema es que, si no tocamos ese valor con `auto-cost reference-bandwidth`, cualquier enlace de 100 Mbps o más nos queda con costo 1, así que OSPF termina sin poder distinguir un enlace de 100 Mbps de uno de 1 Gbps y elige caminos casi como si solo contara saltos. Como en nuestras topologías vamos a tener un enlace deliberadamente angosto (el cuello de

botella que armamos a propósito, Sección 3), vamos a tener que revisar este valor para que el costo realmente refleje esa diferencia.

Para completar el panorama de las rutas estáticas que ya mencionamos en la Tabla 1: cada ruta, estática o aprendida por un protocolo, tiene asociada una distancia administrativa, que es lo que un router usa para decidir cuál preferir cuando aprende la misma ruta por más de una fuente (cuanto más baja, más confiable se la considera; una ruta estática suele tener distancia administrativa 1, más confiable que RIP 120 u OSPF 110). Esto permite armar rutas estáticas flotantes: una ruta estática con distancia administrativa deliberadamente alta, que el router ignora mientras tenga una ruta mejor (por ejemplo, aprendida por OSPF) y que solo entra en uso si esa ruta mejor desaparece – una forma de darle a la red estática algo de respaldo ante una caída sin volverla dinámica.

Por qué vector-distancia converge más lento que estado de enlace

La Tabla 2 ya anticipa que RIP converge más lento que OSPF, pero vale la pena detenerse en el porqué, porque no tiene que ver con qué tan "rápido piensa" cada protocolo sino con qué tan completa es la información con la que arranca a pensar. Un router que corre RIP solo sabe el costo que le reportan sus vecinos directos – no tiene idea de cómo está conectado el resto de la red más allá de ellos –, así que cuando algo cambia, esa novedad tiene que viajar salto a salto: cada router espera (o dispara) una actualización, la procesa, decide si cambió su propia mejor ruta y recién ahí se la pasa al siguiente. Es una cadena de decisiones dependientes, y cada eslabón de esa cadena puede, además, estar temporalmente equivocado con información vieja que todavía no se actualizó – que es exactamente la raíz del problema de *cuenta a infinito* que mencionamos más arriba. Un router OSPF, en cambio, no interpreta nada: cuando recibe un LSA simplemente lo guarda en su base de datos de estado de enlace (LSDB) tal cual lo describe, y una vez que esa base está sincronizada con sus vecinos, calcula su propio camino de punta a punta con Dijkstra sin depender de lo que ningún otro router haya "decidido" mientras tanto. La diferencia de fondo es que RIP propaga conclusiones (mi costo a X es Y) mientras que OSPF propaga hechos (A está conectado a B con costo C), y una conclusión vieja se puede confundir con una nueva, mientras que un hecho viejo simplemente se sobrescribe con uno nuevo.

Para que esto no quede solo en la idea general, vale la pena ver con un ejemplo numérico simple cómo se arma concretamente el problema de cuenta a infinito: una cadena de cuatro routers (A–B–C–D) que originalmente llegan a una red detrás de A con costos 1, 2 y 3 respectivamente. El siguiente cuadro muestra qué pasaría, ronda a ronda, si el enlace A–B se cae y ninguna de las medidas de mitigación que ya mencionamos estuviera activa:

Ronda	Qué sabe B (costo a la red de A)	Qué sabe C (costo a la red de A)
0 (antes de la falla)	1, directo	2, vía B
1 (A–B se cae)	Infinito – B lo detecta directo	2 – todavía no se enteró
2	3 – B escucha a C (2+1) y, sin horizonte dividido, le cree	2 – sin cambios aún
3	3	4 – C escucha a B (3+1) y le cree
4	5 – B vuelve a escuchar a C (4+1)	4
...	... sigue subiendo de a uno por ronda ...	... hasta llegar a 16, el "infinito" de RIP

Ejemplo numérico ilustrativo: cuenta a infinito sin horizonte dividido ni envenenamiento en reversa (no corresponde a una corrida real).

Lo que hace que esta cadena ascendente no ocurra en la práctica es justamente el horizonte dividido con envenenamiento en reversa que describimos más arriba: C nunca le anuncia a B una ruta que aprendió del propio B (o se la anuncia con métrica infinita), así que B nunca tiene la oportunidad de creerle a C una ruta que, en el fondo, salió originalmente de él mismo. Pero esa protección tiene un costo en tiempo: el *hold-down* que la acompaña hace que un router ignore, durante un período prudencial, cualquier anuncio que empeore una ruta recién caída, por si todavía hay información vieja circulando – y ese período de prudencia es, en buena medida, la razón estructural de que RIP necesite varios segundos para converger incluso cuando la actualización disparada funciona bien, mientras que OSPF, al no depender de una cadena de reinterpretaciones sucesivas sino de una única inundación de hechos, no necesita ninguna protección equivalente ni el tiempo de prudencia que eso implica.

El punto clave para nuestro tema

Acá aparece algo que nos parece central para el experimento: ni RIP ni OSPF reaccionan a la congestión. Sus métricas son fijas – RIP cuenta saltos, OSPF usa el ancho de banda nominal del enlace – y ninguna de las dos depende de cuánta carga real está circulando en ese momento. Esto significa que, si saturámos un enlace sin tirarlo abajo, ni RIP ni OSPF van a cambiar de ruta: van a seguir mandando tráfico por el mismo camino saturado, porque desde su punto de vista el enlace sigue arriba y con el mismo costo que antes. Esto cambia lo que podemos esperar de la comparación: la diferencia entre estática, RIP y OSPF que vamos a poder mostrar no es "cuál evita mejor la congestión", sino más bien cómo converge cada una ante una falla y qué tan rápido.

Esta distinción nos sirve para ordenar mejor los ensayos que vamos a hacer. Las pruebas de carga constituyen una parte central del trabajo: en ellas vamos a analizar cómo responden los distintos tipos de tráfico frente a una misma capacidad disponible, observando qué ocurre con la pérdida, el retardo, las retransmisiones y el rendimiento a medida que aumenta la carga. El esquema de encaminamiento se mantendrá sobre el camino seleccionado mientras los enlaces sigan disponibles, lo que permite estudiar los efectos de la congestión sin que un cambio de ruta altere las condiciones del ensayo.

La comparación entre las rutas estáticas, RIP y OSPF va a cobrar sentido principalmente cuando provoquemos la caída de un enlace. Ahí sí cambia el estado de la red y cada esquema tiene que detectar que el camino principal dejó de estar disponible, elegir el respaldo y restablecer la comunicación. Lo que nos interesa medir en ese caso es cuánto dura la interrupción, cuánto tarda en recuperarse el tráfico y qué efecto tiene esa conmutación sobre los flujos que

ya estaban activos.

Mantener constantes la topología, el direccionamiento, las condiciones de carga y el camino de respaldo nos permite que esa comparación sea más clara: si aparecen diferencias en los tiempos de recuperación o en la continuidad de la comunicación, vamos a poder atribuirlas al esquema de encaminamiento y no a que cada ensayo se haya hecho sobre una red distinta.

2.3 Capa de aplicación

El tercer eje tiene que ver con qué protocolos de aplicación vamos a usar para generar el tráfico que después vamos a congestionar. Nos centramos en tres, porque se complementan bien para nuestro objetivo de comparar tráfico reactivo a la congestión contra tráfico que no lo es.

HTTP es el protocolo que vamos a usar para generar el tráfico principal de la simulación. Corre sobre TCP, y acá conviene ser precisos con la versión: en HTTP/1.0 cada solicitud abría y cerraba su propia conexión TCP, así que una página con varios recursos implicaba repetir todo el mecanismo de apertura de conexión – y con él, todo el arranque lento de TCP que desarrollamos en la Sección 2.1 – una y otra vez. HTTP/1.1, que es la versión que vamos a usar como base, soluciona esto con conexiones persistentes: una misma conexión TCP se reutiliza para varias solicitudes seguidas, lo que evita pagar el arranque lento en cada una (Stallings, Sección 22.2). Versiones más nuevas van todavía más lejos: HTTP/2 multiplexa varias solicitudes en paralelo sobre una única conexión TCP, y HTTP/3 directamente deja TCP y corre sobre QUIC, que viaja sobre UDP pero implementa su propio control de congestión a nivel de aplicación en lugar de depender del de TCP. Para nuestro trabajo esto importa porque, al usar HTTP/1.1 sobre TCP clásico, el tráfico que generemos sí va a mostrar los efectos del arranque lento y de la reducción de la ventana ante la congestión – que es justamente lo que nos interesa observar –, algo que con HTTP/3 sería mucho menos directo de interpretar.

El segundo protocolo que nos interesa desarrollar es RTP (*Real-time Transport Protocol*) sobre UDP, porque es el contraste real que nos faltaba frente a HTTP: en los objetivos planteamos comparar tráfico reactivo a la congestión contra un flujo UDP en tiempo real, y hasta ahora solo habíamos desarrollado SNMP, que no sirve para eso porque es tráfico de gestión, no de usuario. RTP es el protocolo que usan las aplicaciones de voz y video (una videollamada, streaming en vivo) para empaquetar audio y video sobre UDP, con número de secuencia y marca de tiempo para que el receptor pueda reordenar y reproducir a un ritmo constante. La diferencia clave con TCP/HTTP es que RTP no retransmite: si se pierde un paquete de audio de hace medio segundo, retransmitirlo no sirve de nada porque para cuando llegue ya pasó el momento de reproducirlo, así que el receptor simplemente lo salta y sigue. Lo que sí le importa a este tipo de tráfico es el *jitter* (la variación en el tiempo de llegada entre paquetes consecutivos), porque si varía demasiado el buffer de reproducción se queda sin datos o se desborda; y tolera cierta pérdida de paquetes, hasta un punto, sin que la calidad percibida se caiga demasiado. Esto es exactamente lo opuesto a cómo reacciona TCP, y es lo que vamos a poder contrastar generando un flujo RTP simulado – por ejemplo con `iperf3` en modo UDP a tasa fija – en paralelo con el tráfico HTTP.

SNMP, por su parte, lo traemos a colación por un motivo distinto. Es el protocolo estándar para la gestión de redes: una estación de gestión consulta o modifica variables (objetos MIB) de los agentes instalados en los dispositivos de la red mediante mensajes `Get`, `Set` y notificaciones asíncronas (*traps*) (Stallings, Sección 22.3). A diferencia de HTTP, SNMP corre sobre UDP, así que no hereda ningún mecanismo de control de congestión: si se pierde un mensaje, simplemente se pierde. Nos interesa mencionarlo porque, en una red real, SNMP sería justamente la herramienta que se usaría para monitorear de forma automática los mismos indicadores de

congestión que nosotros vamos a observar "a mano" con Wireshark (pérdida de paquetes, uso de los enlaces), lo que nos permite conectar nuestra simulación con cómo se manejaría este mismo problema en producción.

En conjunto, estos tres ejes nos dan el marco que necesitábamos para diseñar la parte experimental: sabemos qué mecanismos de congestión vamos a buscar en las capturas, qué protocolos de encaminamiento vamos a comparar y qué tipo de tráfico vamos a generar para poner a prueba todo esto.

3 Metodología

Esta sección describe cómo vamos a generar la congestión, qué vamos a medir y con qué herramienta, y cómo vamos a estructurar el análisis de los resultados.

El cuello de botella

Antes de simular nada nos topamos con un problema práctico: en GNS3, por defecto, los enlaces virtuales no tienen un límite de ancho de banda que permita ubicar la congestión de forma controlada dentro de la topología. Si no configuramos una restricción explícita, el tráfico podría circular a la velocidad que permitan el equipo anfitrión y las interfaces virtuales, sin garantizar que la saturación aparezca en el enlace que queremos estudiar ni que las distintas repeticiones sean comparables. Por eso vamos a configurar una cola de salida en el router sobre el enlace elegido, fijando una capacidad inferior a la carga ofrecida. Así, el cuello de botella estará definido por el diseño del experimento, tanto en su ubicación como en su capacidad, y podremos observar cómo evolucionan la pérdida, el retardo, las retransmisiones y el rendimiento cuando el tráfico supera ese límite.

La licencia gratuita de MikroTik CHR limita la velocidad de cada interfaz a 1 Mbps. Esta condición se tendrá en cuenta al definir los parámetros de los ensayos: la capacidad configurada para el enlace restringido será inferior a ese valor. De este modo, la cola aplicada en el router determinará la capacidad efectiva del enlace bajo estudio.

Generación de tráfico y métricas

Para generar tráfico de prueba entre las puntas de la topología vamos a usar `iperf3`, que nos permite tanto tráfico TCP (para ver el comportamiento de HTTP) como UDP a tasa fija (para simular el flujo RTP en tiempo real que describimos en la Sección 2.3). La Tabla 4 resume qué herramienta usamos para medir cada métrica.

Métrica	**Herramienta**
Throughput TCP	`iperf3` en modo TCP y PCAP analizado con Wireshark
Throughput, jitter y pérdida UDP	`iperf3` en modo UDP con tasa fija (`-b`)
Latencia	`ping` (RTT) durante el ensayo
Arranque lento y caída de *cwnd*	JSON de `iperf3` con `snd_cwnd` y PCAP analizado con Wireshark
Tiempo de convergencia	Evento de falla y logs de rutas activas; `ping` continuo para observar la recuperación del servicio

Tabla 4. Métricas a registrar y herramienta usada para cada una.

De detectar a explicar

No alcanza con mostrar que hay una diferencia entre las capturas de las distintas topologías: hay que explicarla. Para que esto no quede librado a cómo le salga a cada uno, proponemos que cada resultado que presentemos siga siempre la misma estructura de tres pasos: primero, qué observamos concretamente en la captura (el número, el gráfico, el paquete puntual); segundo, qué predecía la teoría que desarrollamos en el marco teórico, con referencia explícita a la sección correspondiente; y tercero, por qué coincide o por qué no – y si no coincide, qué variable de nuestra simulación (por ejemplo, que usamos CUBIC en lugar de Reno, o que no ajustamos el costo de referencia de OSPF) puede explicar la diferencia.

Etapas de trabajo

El trabajo se organizó en dos líneas experimentales complementarias. La primera estudia cómo cambia el comportamiento de los flujos cuando aumenta la carga sobre un enlace con capacidad limitada. La segunda estudia qué ocurre cuando ese enlace deja de estar disponible y la red debe utilizar un camino alternativo. Esta separación permite no confundir los efectos de la congestión con los de un cambio de encaminamiento.

Como base se utilizó una ùnica topología en GNS3, formada por cinco routers y dos extremos de comunicación. La estructura física, el direccionamiento y el cuello de botella se mantuvieron constantes en todos los ensayos. Sobre esa misma red se configuraron, por separado, rutas estáticas, RIPv2 y OSPF. El detalle de la topología y de las configuraciones utilizadas se conserva en los Anexos A y B.

Para los ensayos de congestión se fijó un cuello de botella de 500 Kbit/s sobre el enlace principal. Se generó tráfico UDP a tasas crecientes, desde 100 hasta 800 Kbit/s, para observar el punto en que el rendimiento deja de acompañar a la carga ofrecida y comienzan a crecer la pérdida y el jitter. También se realizaron ensayos de referencia con TCP y HTTP/1.1, y ensayos de contención donde un flujo UDP compite con un flujo TCP o HTTP por la misma capacidad disponible.

Estos ensayos de carga se realizaron sobre el esquema estático. La decisión responde a que los protocolos de encaminamiento no modifican su ruta ante la congestión si el enlace continúa activo: RIP y OSPF mantienen su camino mientras el enlace esté disponible. Por lo tanto, repetir la misma curva de congestión con los otros esquemas no agregaba una hipótesis nueva, mientras que la comparación entre ellos sí resulta relevante ante la caída de un enlace.

En cada condición se realizaron cinco repeticiones y se conservaron los resultados crudos de las herramientas de generación y medición, junto con las capturas de tráfico. Esto permitió comparar las mediciones entre repeticiones y respaldar el análisis con evidencia reproducible. El inventario de las capturas y el registro de las corridas instrumentadas se presentan en los Anexos C y D.

Para evaluar la recuperación ante fallas, se suspendió de forma controlada el enlace principal R2–R4 mientras circulaban tráfico HTTP e ICMP entre los extremos. En paralelo se registraron los eventos de la falla, las respuestas de ping, las capturas sobre el enlace afectado y sobre el camino de respaldo, y las rutas activas de los routers. Así fue posible distinguir entre la recuperación que percibe el cliente y el instante en que la red instala efectivamente el camino alternativo.

Finalmente, los resultados se analizaron relacionando cada observación con los mecanismos desarrollados en el marco teórico. Para cada caso se identificó qué cambió en las métricas, qué comportamiento se esperaba según TCP, UDP o el esquema de encaminamiento activo, y qué características de la emulación ayudan a explicar el resultado.

4 Implementación real y costos

Esta sección le pone nombre, apellido y precio a los equipos, distinguiendo entre lo que usamos gratis para simular en la facultad y lo que costaría armar esto mismo con hardware real.

Alcance: nuestra propia topología, no una red hipotética de empresa

En vez de inventar un escenario de red que no se corresponde con lo que realmente vamos a simular, decidimos cotizar lo que haría falta para armar nuestra propia topología de laboratorio con equipos físicos: dos routers (conectados entre sí por el enlace que usamos como cuello de botella), dos switches (uno de cada lado, para conectar las puntas) y dos computadoras o servidores livianos que hacen de cliente y servidor de `iperf3`. Es el mismo hardware para las tres variantes de encaminamiento que comparamos (estática, RIP, OSPF): en la práctica no haría falta comprarlo tres veces, alcanzaría con reconfigurar los mismos equipos entre corrida y corrida, así que cotizamos un solo juego.

Simulación en GNS3: costo $0

Todo lo que usamos en la computadora para simular es gratuito o de código abierto: GNS3, Wireshark e iperf3/Linux no tienen costo de licencia, y la imagen de router que elegimos, MikroTik CHR, tiene una versión de prueba gratuita que alcanza para simular todas las funciones de ruteo que necesitamos (la única limitación es que capa la velocidad a 1 Mbps por interfaz, que como ya dijimos nos viene bien como cuello de botella). Esto es justamente parte de lo que nos interesa remarcar: la simulación no cuesta nada, pero no refleja el costo de montar la red de verdad.

Equipos reales: tres niveles de calidad

Para la implementación real armamos tres opciones por rol, pensando en el triángulo calidad, tiempo y presupuesto. Los precios son los que cotizó uno de nuestros compañeros en el mercado argentino; los convertimos a pesos con el tipo de cambio oficial del 28 de octubre de 2026 ($1.560 por dólar, fuente: indicadores.ar) para que el número final tenga sentido en nuestro contexto, ya que un precio en dólares sin fecha ni tipo de cambio no dice mucho en la Argentina de hoy.

Rol	Económico	Intermedio	Empresarial
Router de núcleo / borde	MikroTik hEX S o RB750Gr3 (USD 70–120)	Cisco Router serie 900 / Ubiquiti UniFi Gateway (USD 300–600)	Cisco ISR 1100 / Catalyst 8000 (USD 1.500–3.500+, más licencias IOS XE)
Switch	MikroTik CSS326 / TP-Link Smart Managed (USD 100–180)	Cisco Business 250/350 Series (USD 300–500)	Cisco Catalyst 9200/9300 (USD 1.800–4.000+)
Monitoreo (SNMP)	Zabbix / LibreNMS, software libre (USD 0)	Zabbix / LibreNMS (USD 0)	PRTG Network Monitor (desde USD 1.800)

Tabla 5. Equipos por rol y nivel de calidad, para armar nuestra propia topología de laboratorio con hardware real.

Nivel	Total aprox. (USD)	Total aprox. (ARS, $1.560/USD)
Económico	USD 200–350	$312.000 – $546.000
Intermedio	USD 700–1.200	$1.092.000 – $1.872.000
Empresarial	USD 5.000+	$7.800.000+

Tabla 6. Costo total aproximado por nivel, cotizado por el grupo en el mercado argentino (28/10/2026, dólar oficial $1.560 – fuente: indicadores.ar).

Que lo emulado coincida con lo que se compra

No es casualidad que hayamos elegido MikroTik CHR para emular los routers: corre RouterOS, el mismo sistema operativo que traen los routers MikroTik físicos, así que la configuración que armamos en GNS3 (rutas estáticas, RIP, OSPF) es literalmente la misma que usaríamos en el equipo real de nivel económico. Si hubiéramos elegido Cisco IOSv, en cambio, hubiéramos necesitado una licencia paga (CML) para emularlo, y los equipos Cisco reales son más caros e incluyen contratos de soporte – lo cual tiene sentido para el nivel empresarial, pero no para validar nuestra topología de laboratorio.

Monitoreo

Ya mencionamos SNMP en la Sección 2.3 como el protocolo que se usaría en producción para monitorear automáticamente los mismos indicadores que nosotros observamos "a mano" con Wireshark. Acá también hay una decisión de costo: herramientas de monitoreo libres como Zabbix o LibreNMS no tienen costo de licencia y alcanzan perfectamente para una red de nuestro tamaño, mientras que una herramienta paga como PRTG Network Monitor (desde aproximadamente USD 1.800 de licencia) suma funcionalidades pensadas para redes mucho más grandes, que para nuestro caso serían un gasto innecesario.

Calidad, tiempo y presupuesto

Cerramos con cómo pensamos ese equilibrio entre calidad, tiempo y presupuesto, aplicado a nuestro propio caso:

- La **ruta estática** es la opción más barata en equipos – no necesita routers con soporte de protocolos dinámicos avanzados – pero la más cara en horas de operación: cada cambio en la red hay que cargarlo a mano, y un error humano tarda en notarse.
- **OSPF** necesita personal más capacitado para configurarlo y mantenerlo bien (áreas, costos, autenticación), pero una vez armado escala mejor y converge mucho más rápido que RIP ante una caída.
- Para nuestro propio escenario de laboratorio, el nivel económico (USD 200–350) ya alcanza para validar todo lo que planteamos: los tres esquemas de encaminamiento, el cuello de botella y las capturas. El nivel intermedio (USD 700–1.200) tendría sentido recién si pensamos en una red que vaya a seguir creciendo después del laboratorio; el nivel empresarial (USD 5.000+) queda totalmente fuera de escala para lo que necesitamos nosotros, pero lo dejamos documentado porque muestra hasta dónde llega el mismo problema de encaminamiento cuando la red es mucho más grande.

5 Resultados y análisis

Esta sección presenta las 80 corridas instrumentadas de la matriz de experimentos. Los resultados se derivan de los JSON de `iperf3`, las salidas de `wrk`, los logs de `ping`, los eventos, los logs de rutas activas y los PCAP de cada corrida. Donde reportamos una mediana, corresponde a las cinco repeticiones de la condición indicada.

El alcance no es parejo entre esquemas: la curva de congestión, las referencias TCP y HTTP, la contención y el arranque lento se midieron con rutas estáticas. Las fallas se midieron con rutas estáticas, RIP y OSPF. Por eso los ensayos de carga caracterizan el cuello en el escenario estático, mientras que la comparación entre esquemas se limita a la recuperación ante falla. La Tabla 7 resume esta cobertura.

Ensayo	Estática	RIP	OSPF
Curva de congestión UDP (8 tasas, 5 repeticiones)	Completo (40 corridas)	No priorizado*	No priorizado*
TCP y HTTP/1.1 de referencia	Completo (10 corridas)	No medido	No medido
Contención TCP+UDP y HTTP+UDP	Completo (10 corridas)	No medido	No medido
Arranque lento TCP (5 repeticiones)	Completo	No medido	No medido
Convergencia ante falla (5 repeticiones)	Completo	Completo	Completo

Tabla 7. Cobertura de las corridas instrumentadas por esquema de encaminamiento.

Curva de congestión UDP

Qué observamos. Hicimos cinco repeticiones de 60 segundos para cada una de ocho tasas de UDP ofrecidas (100K a 800K), siempre contra el mismo cuello de 500 Kbit/s en `R2-R4`. La Tabla 8 muestra la mediana de throughput, jitter y pérdida recibidos en cada tasa; agregamos también qué porcentaje de la tasa ofrecida efectivamente se retuvo, para que el punto de quiebre de la curva se lea de un vistazo sin tener que calcularlo.

Tasa ofrecida	Throughput mediano	Jitter mediano	Pérdida mediana	% de la tasa retenido
100 Kbit/s	99,993 Kbit/s	1,445 ms	0,000 %	100,0 %
200 Kbit/s	199,991 Kbit/s	1,741 ms	0,000 %	100,0 %
300 Kbit/s	299,972 Kbit/s	1,246 ms	0,000 %	100,0 %
400 Kbit/s	399,973 Kbit/s	1,945 ms	0,000 %	100,0 %
500 Kbit/s	416,643 Kbit/s	6,086 ms	16,576 %	83,3 %
600 Kbit/s	438,725 Kbit/s	9,209 ms	26,613 %	73,1 %
700 Kbit/s	423,077 Kbit/s	8,589 ms	39,337 %	60,5 %
800 Kbit/s	426,811 Kbit/s	8,101 ms	46,620 %	53,4 %

Tabla 8. Curva de congestión UDP, esquema estático (mediana de 5 repeticiones por tasa). El color de la última columna va de verde (retención total) a rojo (mayor pérdida).

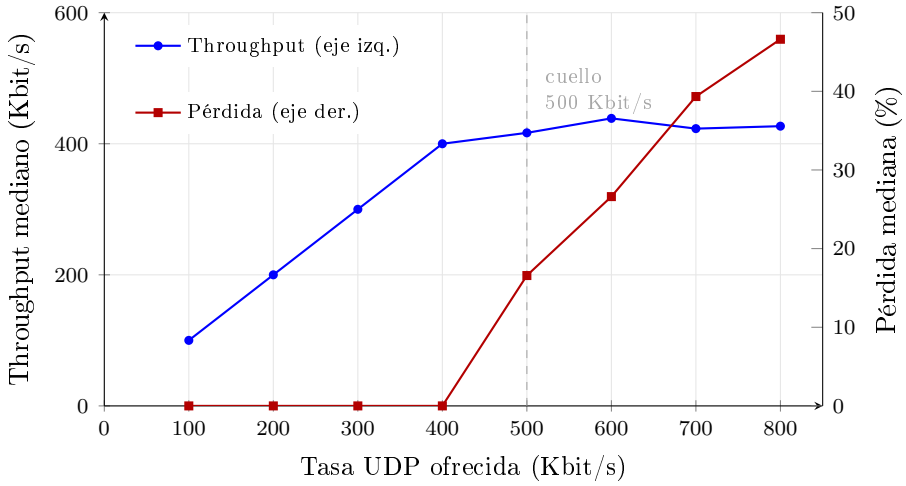


Gráfico 3. Curva real de congestión UDP, esquema estático: throughput mediano (eje izquierdo, azul) y pérdida mediana (eje derecho, rojo) en función de la tasa ofrecida (datos de la Tabla 8).

Qué predecía la teoría. En la Sección 2.1 describimos, siguiendo a Stallings, que el throughput real debería acompañar linealmente a la carga ofrecida hasta un punto, y después entrar primero en congestión moderada (el throughput sigue creciendo pero cada vez más lento) y después severa (el throughput deja de crecer o cae), tal como ilustra el Gráfico 1; el Gráfico 3 es la versión con datos reales de esa misma forma.

Por qué coincide. La teoría anticipaba este resultado porque una cola puede atender el tráfico sin acumular paquetes mientras la tasa de llegada no supera la capacidad de salida del enlace. Por eso, hasta 400 Kbit/s, el throughput recibido acompaña prácticamente a la tasa ofrecida y no aparecen pérdidas: los paquetes pueden salir de la cola antes de que esta se complete.

Al llegar a 500 Kbit/s y superarlo, la tasa ofrecida alcanza o excede la capacidad configurada del cuello de botella. En esa situación, los paquetes ingresan a la cola más rápido de lo que pueden transmitirse por el enlace, la ocupación aumenta y también lo hace el tiempo de espera. Cuando la cola se llena, los paquetes que no pueden ser almacenados se descartan, lo que explica el aumento de pérdida y la variación del retardo observada en los datagramas que sí llegan.

UDP no reduce automáticamente su tasa de envío ante esas pérdidas, a diferencia de TCP. Por eso el emisor continúa ofreciendo la misma carga aunque el enlace ya esté saturado: el throughput recibido deja de crecer en la misma proporción que la tasa enviada, mientras la pérdida aumenta a medida que se incrementa la carga. El cambio observado desde 500 Kbit/s es, entonces, el comportamiento esperado de un flujo UDP frente a una cola limitada por debajo de la carga ofrecida.

TCP y HTTP/1.1 de referencia

Qué observamos. Para el flujo TCP de referencia (`iperf3`, sin tráfico competidor) corrimos cinco repeticiones de 60 segundos contra el mismo cuello; la Tabla 9 muestra el detalle por corrida. Para HTTP/1.1 generamos tráfico real con Nginx sirviendo un recurso de 256 KiB y `wrk` solicitándolo desde el cliente; la Tabla 10 resume las cinco corridas.

Corrida	Throughput	Retransmisiones	RTT TCP medio
r01	407,798 Kbit/s	337	70,934 ms
r02	451,115 Kbit/s	389	70,035 ms
r03	408,656 Kbit/s	323	70,946 ms
r04	435,133 Kbit/s	398	66,400 ms
r05	393,791 Kbit/s	382	62,296 ms
Mediana	**408,656 Kbit/s**	**382**	**70,035 ms**

Tabla 9. TCP de referencia, esquema estático (`iperf3`, sin competencia).

Corrida	Latencia	Solicitudes/s	Transferencia
r01	5,39 s	0,18	47,56 KB/s
r02	5,49 s	0,17	46,57 KB/s
r03	5,41 s	0,18	47,42 KB/s
r04	5,44 s	0,18	47,14 KB/s
r05	5,46 s	0,17	46,69 KB/s
Mediana	**5,44 s**	**0,18**	**47,14 KB/s**

Tabla 10. HTTP/1.1 de referencia, esquema estático (Nginx + `wrk`, recurso de 256 KiB).

Qué predecía la teoría. Para TCP, la Sección 2.1 anticipaba que, ante un enlace angosto, iban a aparecer retransmisiones y un RTT inflado por la espera en cola, con CUBIC (no Reno/Tahoe) marcando la forma exacta de esa reacción. Para HTTP, la Sección 2.3 anticipaba que, al usar HTTP/1.1 con conexiones persistentes, el efecto dominante del cuello iba a ser el throughput disponible y no el arranque lento repetido en cada solicitud.

Por qué coincide. Cuando TCP detecta pérdidas o aumentos sostenidos de RTT, interpreta que la red está congestionada y reduce la cantidad de datos que puede mantener en vuelo. Esa reacción limita el throughput mientras el flujo intenta volver a aumentar su ventana; si la cola sigue bajo presión, las retransmisiones y las demoras vuelven a aparecer. Por eso las corridas muestran simultáneamente retransmisiones, RTT elevado y un throughput inferior a la capacidad nominal del cuello: parte de la capacidad se consume en encabezados, confirmaciones y reenvíos, y parte del tiempo el emisor transmite con una ventana reducida.

HTTP/1.1 usa ese mismo transporte TCP. Como la conexión se mantiene abierta para sucesivas solicitudes, el recurso de 256 KiB queda condicionado principalmente por el throughput que TCP puede sostener sobre el cuello. Una menor transferencia efectiva implica que cada respuesta demora más en completarse, por lo que disminuyen las solicitudes por segundo y aumenta la latencia que informa `wrk`. La serie independiente de arranque lento aporta, además, la evidencia directa sobre la evolución inicial de `snd_cwnd`.

Arranque lento y evolución de *cwnd*

Qué observamos. Ejecutamos cinco flujos TCP de 60 segundos sin período omitido y con intervalos de 0,2 segundos. Cada JSON conserva 300 intervalos desde el inicio de la conexión, el valor de `snd_cwnd`, el RTT, las retransmisiones y el throughput de cada intervalo. Esta serie permite observar la evolución inicial de la ventana sin inferirla a partir de una corrida que haya descartado sus primeros segundos.

Qué predecía la teoría. Al comenzar una conexión TCP, el emisor aumenta la cantidad de datos que mantiene en vuelo hasta encontrar una señal de congestión. Las pérdidas o los aumentos de retardo reducen esa ventana y el emisor vuelve a incrementarla según el algoritmo

de control de congestión activo.

Por qué coincide. Los intervalos conservados muestran que `snd_cwnd` no es una capacidad fija del enlace, sino una variable que el emisor ajusta mientras compite con el cuello de 500 Kbit/s. El crecimiento inicial y los ajustes posteriores se interpretan junto con RTT y retransmisiones de la misma serie. Como los endpoints usan CUBIC, la forma exacta de la curva no tiene por qué reproducir el dibujo clásico de Tahoe o Reno; la evidencia permite observar el mecanismo real del transporte utilizado en la emulación.

Contención TCP+UDP y HTTP+UDP

Qué observamos. En estos ensayos arrancamos un flujo UDP a tasa fija (400K) y, a los 10 segundos, sumamos un segundo flujo – TCP de referencia en un caso, HTTP/1.1 en el otro – compitiendo por el mismo cuello durante el resto de la corrida. Las Tablas 11 y 12 muestran el detalle de las cinco repeticiones de cada ensayo, agregando en cada una el total combinado de los dos flujos (expresando el HTTP en Kbit/s equivalentes) para poder comparar ese total contra el cuello de 500 Kbit/s.

Corrida	TCP	UDP	Pérdida UDP	Jitter UDP	Total
r01	113,8 Kbit/s	278,0 Kbit/s	30,44 %	12,56 ms	391,8 Kbit/s
r02	90,0 Kbit/s	288,4 Kbit/s	27,84 %	15,13 ms	378,5 Kbit/s
r03	141,5 Kbit/s	253,4 Kbit/s	36,44 %	21,69 ms	394,9 Kbit/s
r04	120,1 Kbit/s	269,0 Kbit/s	32,44 %	8,40 ms	389,1 Kbit/s
r05	231,4 Kbit/s	249,8 Kbit/s	37,28 %	13,58 ms	481,2 Kbit/s
Mediana	**120,1 Kbit/s**	**269,0 Kbit/s**	**32,44 %**	**13,58 ms**	**391,8 Kbit/s**

Tabla 11. Contención TCP+UDP, esquema estático (UDP a 400K desde los 0 s, TCP desde los 10 s). Rojo: flujo que cede ancho de banda (TCP); azul: flujo que lo retiene (UDP).

Corrida	Latencia HTTP	Transf. HTTP	UDP recibido	Pérdida UDP	Jitter UDP	Total equiv.
r01	13,92 s	17,67 KB/s	249,0 Kbit/s	37,62 %	8,21 ms	390,3 Kbit/s
r02	20,60 s	13,65 KB/s	272,2 Kbit/s	31,91 %	17,11 ms	381,4 Kbit/s
r03	20,79 s	12,74 KB/s	268,0 Kbit/s	32,91 %	10,44 ms	369,9 Kbit/s
r04	15,58 s	16,65 KB/s	258,2 Kbit/s	35,35 %	11,40 ms	391,4 Kbit/s
r05	22,89 s	11,25 KB/s	276,3 Kbit/s	30,87 %	9,46 ms	366,3 Kbit/s
Mediana	**20,60 s**	**13,65 KB/s**	**268,0 Kbit/s**	**32,91 %**	**10,44 ms**	**381,4 Kbit/s**

Tabla 12. Contención HTTP+UDP, esquema estático (UDP a 400K en reversa desde los 0 s, HTTP desde los 10 s). Total equivalente: Kbit/s de UDP más la transferencia HTTP expresada en Kbit/s (KB/s × 8). Rojo: HTTP cede; azul: UDP retiene.

Qué predecía la teoría. En la Sección 2.1 mencionamos la imparcialidad como uno de los criterios de gestión de tráfico que describe Stallings: en un esquema imparcial, todos los flujos deberían sufrir la congestión en una medida parecida. También anticipamos, en la Sección 2.3, que UDP no tiene ningún mecanismo de control de congestión – sigue mandando a tasa fija pase lo que pase –, mientras que TCP y HTTP sí reaccionan bajando su rendimiento efectivo.

Por qué coincide. Al iniciarse el segundo flujo, los dos emisores pasan a competir por una capacidad que ya es insuficiente para transportar toda la carga ofrecida. UDP continúa enviando a 400 Kbit/s porque no modifica automáticamente su tasa ante pérdidas, mientras que TCP reacciona a las señales de congestión reduciendo su ventana de envío. HTTP hereda esa reacción porque sus respuestas viajan sobre TCP. Como consecuencia, UDP conserva una porción mayor del throughput recibido y los flujos TCP/HTTP reducen su rendimiento

efectivo; al mismo tiempo, la pérdida y el jitter de UDP muestran que también ese flujo sufre la saturación.

La diferencia no implica que UDP quede aislado de la congestión ni que el reparto sea constante entre repeticiones. En TCP+UDP el recurso compartido es el cuello R2–R4; en HTTP+UDP es el cuello inverso R4–R2. La variabilidad observada es esperable porque el instante de llegada de los paquetes, las retransmisiones y la ocupación de la cola cambian durante cada corrida.

Convergencia ante falla

Qué observamos. En las cinco repeticiones de cada esquema suspendimos R2–R4 a los 20 segundos, con HTTP, ping continuo, eventos, PCAP y monitoreo de rutas activas en paralelo. La convergencia de reenvío se mide desde el evento hasta que R1 y R4 muestran el gateway de respaldo. Como las rutas se sondean aproximadamente cada 200 ms, cada corrida se informa como un intervalo: la cota inferior es la ùltima muestra anterior al cambio y la superior la primera muestra que confirma el respaldo. El ping se conserva por separado como recuperación observable del servicio. La Tabla 13 resume ambas mediciones.

Esquema	Convergencia de reenvío, mediana	Rango de cotas superiores	Recup. ICMP mediana
Estática	[24,928; 25,131] s	22,083–31,142 s	24,986 s
RIP	[1,145; 20,802] s	6,415–24,395 s	19,007 s
OSPF	[1,853; 2,054] s	1,343–2,214 s	1,959 s

Tabla 13. Convergencia de reenvío observada y recuperación ICMP ante falla ($n = 5$ por esquema). Las cotas no son tiempos exactos de convergencia.

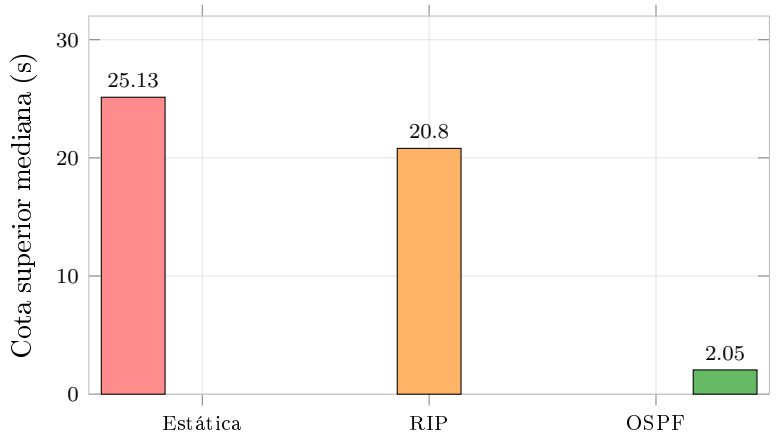


Gráfico 4. Cota superior mediana de convergencia de reenvío por esquema (datos de la Tabla 13).

El detalle de las cotas de ruta para el esquema estático se muestra en la Tabla 14. La Tabla 15 detalla la recuperación ICMP observada para RIP como indicador complementario de servicio, no como tiempo de convergencia.

Corrida	Cota inferior	Cota superior	Recup. ICMP
r01	24,928 s	25,131 s	25,009 s
r02	30,941 s	31,142 s	30,979 s
r03	21,882 s	22,083 s	22,062 s
r04	23,056 s	23,257 s	23,035 s
r05	25,034 s	25,237 s	24,986 s
Mediana	**24,928 s**	**25,131 s**	**24,986 s**

Tabla 14. Acotación de la convergencia de reenvío y recuperación ICMP, rutas estáticas.

Corrida	Último ping antes de la falla	Reanudación sostenida	Indisponibilidad
r01	0,783 s	6,415 s	6,284 s
r02	1,145 s	24,395 s	24,127 s
r03	1,916 s	20,802 s	18,886 s
r04	1,859 s	19,154 s	19,007 s
r05	1,054 s	23,268 s	23,040 s
Mediana	**1,145 s**	**20,802 s**	**19,007 s**

Tabla 15. Recuperación ICMP observada durante la falla, RIP.

Las cotas de RIP son considerablemente más anchas y variables que las de los otros esquemas. Esto exige informar el intervalo observado y no reducir cada corrida a un instante exacto. La recuperación ICMP puede adelantarse o retrasarse respecto de la muestra de ruta disponible; por eso complementa, pero no sustituye, a la convergencia de reenvío.

Qué predecía la teoría. La teoría indica que los esquemas de encaminamiento difieren ante cambios de topología por la forma en que detectan la falla y propagan una ruta alternativa. La comparación relevante, por lo tanto, no es la congestión sino el momento en que cada extremo instala el camino de respaldo.

Por qué coincide. Al suspender R2–R4, el camino principal deja de ser utilizable y los routers deben reemplazarlo por R1–R3–R5–R4. Los logs de rutas muestran directamente ese cambio en ambos extremos. OSPF presenta las cotas más bajas y estables porque el cambio de estado del enlace se propaga mediante su mecanismo de estado de enlace y habilita el recálculo del camino alternativo. Las rutas estáticas dependen del monitoreo activo `check-gateway=ping` para declarar inalcanzable el siguiente salto, por lo que agregan una etapa de detección antes de elegir el respaldo. RIP queda entre ambos, pero sus intervalos son más amplios; las mediciones muestran esa variabilidad sin permitir asignar cada demora a un mensaje o temporizador particular. El ping confirma que el servicio vuelve a ser observable, aunque su recuperación no se usa para definir la convergencia del encaminamiento.

Conclusiones

Las corridas instrumentadas permiten extraer tres conclusiones dentro del alcance del diseño: cómo responde el cuello estático a una carga UDP creciente, cómo interactúan UDP y flujos reactivos al competir por ese recurso y cómo cambia el reenvío ante una falla según el esquema de encaminamiento. Las dos primeras corresponden al escenario estático; la tercera compara rutas estáticas, RIP y OSPF.

La congestión no es un evento, es una curva – y la cruzamos exactamente donde la cola lo permite

El Gráfico 3 y la Tabla 8 muestran con claridad el punto en que el enlace deja de poder acompañar la carga ofrecida. Hasta 400 Kbit/s, el throughput recibido sigue prácticamente a la tasa enviada y la pérdida es nula en las cinco repeticiones de cada condición. A partir de 500 Kbit/s, que coincide con el límite configurado en el cuello R2–R4, el rendimiento ya no crece en la misma proporción: la retención baja a 83,3 % en 500 Kbit/s y continúa descendiendo hasta 53,4 % en 800 Kbit/s, donde la pérdida mediana alcanza 46,6 %.

La transición no es un cambio arbitrario en el generador de tráfico, sino la consecuencia de ofrecer al enlace más datos de los que puede transmitir. Mientras la tasa de llegada permanece por debajo de la capacidad de salida, la cola puede vaciarse entre paquetes y no necesita descartar. Cuando la carga alcanza el límite, los paquetes empiezan a acumularse; al agotarse el espacio disponible, los nuevos datagramas no pueden esperar indefinidamente y se pierden. Por eso, a medida que aumentamos la tasa ofrecida por encima del cuello, el throughput recibido deja de representar una fracción constante de esa tasa y la pérdida pasa a ser la señal dominante de saturación.

El resultado también permite distinguir el comportamiento de UDP del de un transporte reactivo. El emisor UDP no reduce por sí mismo la tasa configurada cuando detecta o sufre pérdidas: continúa ofreciendo 500, 600, 700 u 800 Kbit/s aunque el cuello solo pueda transportar una parte. La red absorbe entonces la diferencia mediante encolamiento y descarte. Esa es la razón por la que la curva conserva una zona lineal al principio y luego muestra throughput limitado, jitter mayor y pérdida creciente, exactamente los indicadores que buscábamos observar al imponer un cuello de botella controlado.

El jitter completa esa lectura. En la zona sin saturación las variaciones de retardo son bajas porque los paquetes atraviesan el cuello sin permanecer demasiado tiempo en la cola. Una vez que la carga alcanza la capacidad disponible, el tiempo de espera empieza a depender de cuántos paquetes llegaron antes y de cuánto espacio queda por transmitir; por eso el jitter aumenta y no necesariamente lo hace de forma perfectamente lineal entre una tasa y la siguiente. A tasas altas parte de la carga deja de convertirse en espera y pasa directamente a convertirse en pérdida, porque los datagramas ya no encuentran lugar en la cola.

Esto es importante para un uso de tiempo real. Una aplicación de voz o video puede tolerar cierta pérdida mejor que una retransmisión tardía, pero necesita que los paquetes que sí llegan lo hagan con una cadencia suficientemente regular. El aumento conjunto de jitter y pérdida muestra que el problema no es solo cuántos bits por segundo llegan al receptor: también importa cuándo llega cada datagrama. UDP no resuelve esa adaptación a nivel de transporte; la aplicación o la red deben decidir cómo responder a esas señales de degradación.

Compartir un enlace no es repartirlo: lo que separa a UDP de TCP y HTTP bajo presión

Las Tablas 11 y 12 muestran la asimetría que anticipamos al hablar de imparcialidad, pero los valores registrados permiten describirla con más cuidado que una simple oposición entre "UDP gana y TCP pierde". En TCP+UDP, el flujo TCP entrega una mediana de 120,1 Kbit/s, mientras UDP recibe 269,0 Kbit/s; ambos están por debajo de lo que ofrecían sin competencia, pero la reducción no tiene la misma magnitud. En HTTP+UDP ocurre el mismo patrón desde la capa de aplicación: UDP recibe 268,0 Kbit/s, mientras la transferencia HTTP cae a 13,65 KB/s y la latencia mediana de cada solicitud llega a 20,60 s. La competencia no elimina a UDP de la congestión –su pérdida y su jitter también aumentan–, pero el flujo reactivo termina cediendo una proporción mayor de su rendimiento.

Esa diferencia tiene una explicación en el modo en que los dos transportes responden a la misma señal de congestión. Cuando la cola empieza a descartar paquetes o a aumentar el retardo, TCP interpreta esas señales como indicios de que está enviando más de lo que la red puede transportar. Reduce entonces su ventana de congestión, limita la cantidad de datos en vuelo y necesita tiempo para volver a aumentar su ritmo. UDP, en cambio, no incorpora ese mecanismo de control de congestión: el emisor continúa ofreciendo los 400 Kbit/s configurados aunque una parte creciente de los datagramas no alcance el receptor.

El efecto se acumula durante la corrida. La pérdida que comparten los flujos al llegar a la cola limita la entrega de ambos, pero TCP agrega una segunda respuesta en el emisor: retransmite, reduce su ventana y deja de ofrecer la misma cantidad de datos que al comienzo. UDP no agrega esa reducción; su throughput disminuye principalmente porque se descartan datagramas, no porque el emisor se autolimite. De ahí que el reparto observado no sea parejo aun cuando los dos servicios comparten el mismo recurso limitado. No afirmamos con esto que la cola por sí sola explique cada diferencia entre repeticiones: los instantes de llegada, las retransmisiones y la ocupación de la cola también intervienen. Sí muestra, sin embargo, el contraste central que buscaba el ensayo entre un flujo que coopera con el control de congestión y otro que no lo hace a nivel de transporte.

El throughput combinado mediano fue de 391,8 Kbit/s en TCP+UDP y de 381,4 Kbit/s en HTTP+UDP. Por lo tanto, en estas corridas la capacidad ùtil observada quedó por debajo del límite configurado; la interpretación se limita al comportamiento medido en esta topología y estas condiciones de carga.

Este resultado se puede llevar a una situación cotidiana sin confundirla con una reproducción exacta de nuestra topología. Pensemos en una conexión limitada donde, al mismo tiempo, una persona participa de una videollamada y descarga un archivo grande. La aplicación de tiempo real necesita seguir enviando paquetes con regularidad porque un paquete de audio o video que llega demasiado tarde ya no sirve; si usa UDP, no tiene un mecanismo de transporte que reduzca automáticamente su ritmo cuando la red empieza a perder paquetes. La descarga, en cambio, usa TCP: cuando aparecen pérdidas o demoras grandes, TCP interpreta que hay congestión, reduce su ventana y baja la cantidad de datos que mantiene en vuelo.

El resultado visible puede parecer contraintuitivo. La videollamada no queda protegida ni libre de problemas: también puede sufrir pérdidas, cortes breves o variación del retardo. Pero sigue ofreciendo tráfico, mientras que la descarga reacciona reduciendo su propio ritmo para no agravar la congestión. Por eso, bajo una misma capacidad limitada, el flujo no reactivo puede conservar una porción mayor del throughput recibido y el flujo reactivo puede quedar mucho más afectado. Esa es la lectura práctica del contraste que muestran las Tablas 11 y 12: no que UDP "gane" sin costo, sino que TCP paga el costo de cooperar con el control de congestión y

UDP no lo hace a nivel de transporte.

Naturalmente, una conexión hogareña real puede tener otra capacidad, otros algoritmos de cola, más flujos y aplicaciones que adapten su tasa por cuenta propia. El ejemplo no pretende afirmar que toda videollamada y toda descarga se comporten exactamente como nuestras corridas, sino mostrar por qué el mecanismo observado es relevante fuera del laboratorio.

El protocolo de ruteo no evita la congestión, pero decide cuánto dura apagada la red

La Tabla 13 y el Gráfico 4 comparan la convergencia de reenvío observada, no la indisponibilidad ICMP. OSPF presenta las cotas superiores más bajas y estables; las rutas estáticas presentan valores mayores, y RIP muestra un comportamiento intermedio con variabilidad amplia. Esta conclusión se apoya en el cambio de gateway registrado en R1 y R4 y debe interpretarse con la resolución temporal del monitoreo de rutas.

El resultado permite poner en contexto la diferencia entre los tres esquemas sin confundirla con una comparación de indisponibilidad medida solamente desde el cliente. La ruta estática no conoce por sí misma el estado del enlace remoto: necesita que el monitoreo activo `check-gateway=ping` declare inalcanzable el siguiente salto antes de seleccionar el camino de mayor distancia. Esa etapa de detección explica por qué, en nuestras corridas, el respaldo tarda más en instalarse que con los protocolos dinámicos.

OSPF parte de otra lógica: cuando el enlace cambia de estado, los routers propagan información de estado de enlace y recalculan el camino hacia la red remota. Los logs de R1 y R4 muestran que ambos extremos instalan el respaldo dentro de cotas bajas y estrechas, lo que hace que la convergencia observada sea estable entre repeticiones. RIP también aprende y reemplaza rutas dinámicamente, pero las cotas medidas presentan una amplitud mucho mayor. Esto no permite afirmar que cada demora individual provenga de un temporizador o de un mensaje puntual, pero sí muestra que la propagación del cambio fue menos predecible en estas cinco corridas que en OSPF.

El ping continuo aporta una segunda perspectiva que resulta ùtil para interpretar el resultado. Cuando vuelve a responder de forma sostenida, el cliente puede volver a intercambiar tráfico con el servidor; sin embargo, ese instante no reemplaza a la convergencia de reenvío, porque depende del período de sondeo y de los paquetes que logran atravesar la red. Por eso reportamos el cambio de gateway como métrica principal del encaminamiento y la recuperación ICMP como evidencia complementaria de disponibilidad observada desde el extremo.

Relación con los objetivos

Los objetivos específicos se cumplen dentro del alcance definido para cada serie. La topología se configuró y validó con rutas estáticas, RIP y OSPF, manteniendo la misma estructura y el mismo cuello de botella. La carga UDP creciente permitió registrar pérdida, jitter, throughput y RTT a medida que el enlace se aproxima y supera su capacidad. Las referencias TCP y HTTP, junto con las dos pruebas de contención, permitieron contrastar un flujo reactivo con un flujo UDP a tasa fija sobre la misma capacidad limitada.

La comparación entre esquemas de encaminamiento se concentra en el objetivo para el cual se realizaron mediciones en los tres casos: la recuperación ante la caída de R2–R4. Los logs de rutas permiten observar cuándo R1 y R4 instalan el camino de respaldo, mientras que el ping continuo registra cuándo esa conmutación vuelve a ser visible para el cliente. De esta manera, el informe no atribuye a los esquemas de ruteo una curva de congestión que no se midió con

los tres, pero sí compara directamente su respuesta ante la falla que forma parte del diseño experimental.

Limitaciones

Nos parece importante dejar explícitas las limitaciones de trabajar con una simulación y no con una red real, porque condicionan qué tan lejos se puede generalizar lo que medimos:

- La licencia gratuita de MikroTik CHR limita cada egreso a 1 Mbit/s, lo que nos obligó a fijar el cuello experimental por debajo de ese valor (500 Kbit/s) para que la saturación observada no se confundiera con el límite propio de la plataforma.
- La topología es chica – cinco routers en diamante –, pensada para que el experimento sea manejable y no para representar una red de producción; la transición entre la zona sin pérdida y la zona de saturación podría adoptar otra forma en una red con más puntos de buffer intermedios.
- El tráfico TCP de `iperf3` es un flujo de referencia, no HTTP real; lo complementamos con Nginx y `wrk` para tener también tráfico HTTP/1.1 genuino, pero seguimos sin un flujo de tiempo real tipo RTP.
- La implementación de TCP de los endpoints usa CUBIC (Linux), no los algoritmos clásicos (Tahoe/Reno) que desarrollamos en el marco teórico, lo cual ya anticipamos que iba a cambiar la forma exacta de las curvas de *cwnd* – aunque no la dirección del efecto que medimos.
- Los recursos del equipo que corre la simulación acotan cuántas corridas y repeticiones pudimos hacer en paralelo.

Bibliografía

A lo largo del marco teórico (Sección 2) citamos puntualmente cada fuente junto a la idea que respalda; esta sección reúne esas mismas fuentes en un listado único, que vamos a mantener actualizado a medida que incorporemos nuevas referencias en las secciones de Resultados y Conclusiones.

[1] Cisco Systems. *Enhanced Interior Gateway Routing Protocol (EIGRP)* – documentación técnica oficial. Disponíble en: `cisco.com`.

[2] indicadores.ar. *Cotización del dólar oficial*, consultada el 28 de octubre de 2026.

[3] Internet Engineering Task Force. *RFC 2328 – OSPF Version 2.* Disponíble en: `https://www.rfc-editor.org/rfc/rfc2328`.

[4] Internet Engineering Task Force. *RFC 2453 – RIP Version 2.* Disponíble en: `https://www.rfc-editor.org/rfc/rfc2453`.

[5] Internet Engineering Task Force. *RFC 3168 – The Addition of Explicit Congestion Notification (ECN) to IP.* Disponíble en: `https://www.rfc-editor.org/rfc/rfc3168`.

[6] Internet Engineering Task Force. *RFC 4271 – A Border Gateway Protocol 4 (BGP-4).* Disponíble en: `https://www.rfc-editor.org/rfc/rfc4271`.

[7] Internet Engineering Task Force. *RFC 6633 – Deprecation of ICMP Source Quench Messages.* Disponíble en: `https://www.rfc-editor.org/rfc/rfc6633`.

[8] Kurose, J. F. y Ross, K. W. *Redes de computadoras: un enfoque descendente.* Pearson Educación.

[9] Stallings, W. *Comunicaciones y redes de computadores.* Pearson Educación. Capítulo 12, "Encaminamiento en redes conmutadas" (págs. 379–402); Capítulo 13, "Congestión en redes de datos" (págs. 407–441); Capítulo 20, control de congestión en TCP; Capítulo 22, "Aplicaciones distribuidas" (pág. 763 en adelante).

Anexos

Esta sección reúne el material de respaldo de la parte experimental: la topología armada en GNS3, las configuraciones de los routers y el inventario de capturas y corridas. Donde el detalle completo no entra en una tabla del informe, lo resumimos acá.

Anexo A – Topología de red en detalle

Los tres esquemas comparten exactamente la misma topología física y el mismo direccionamiento; lo único que cambia entre ellos es la configuración de ruteo de cada router (Anexo B). El Diagrama 1 muestra la topología en diamante: el camino primario Cliente–R1–R2–R4–Servidor, con el cuello de botella de 500 Kbit/s sobre el enlace R2–R4, y el camino de respaldo R1–R3–R5–R4.

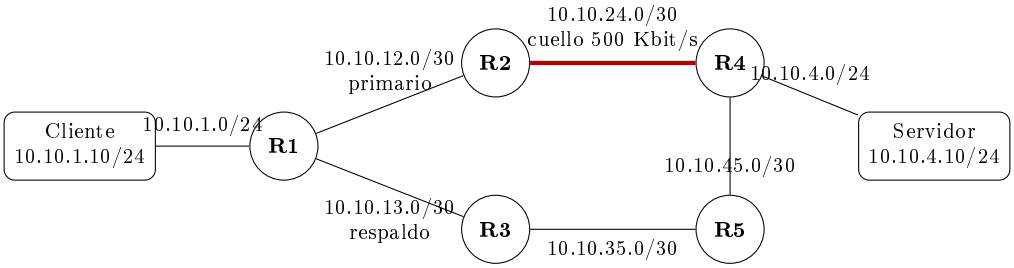
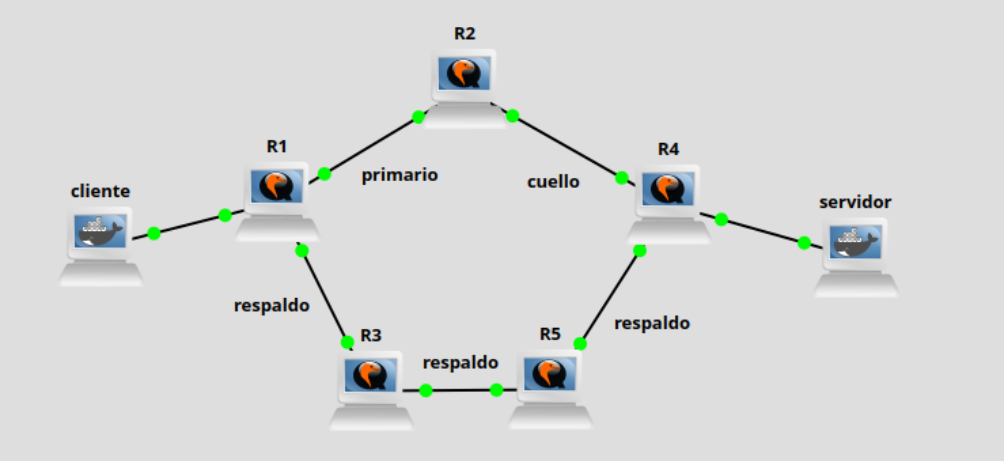


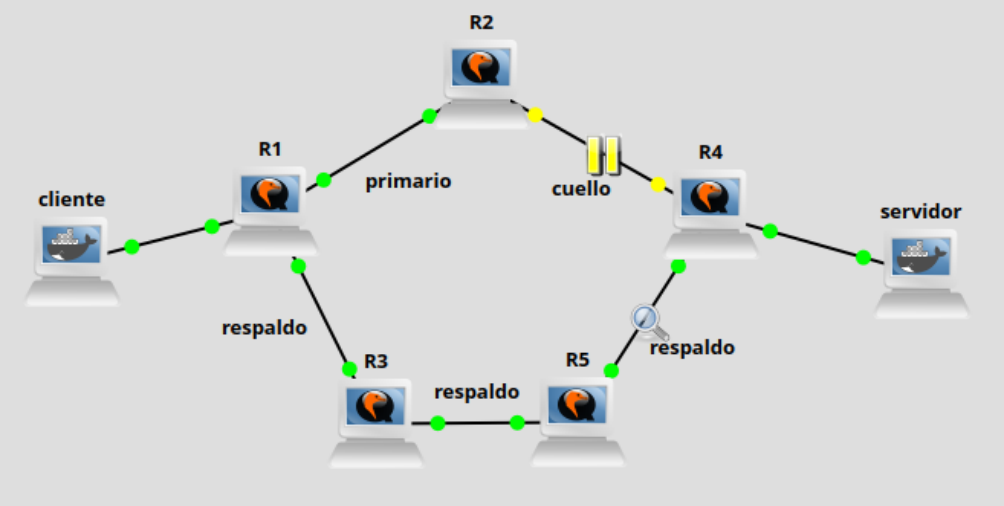
Diagrama 1. Topología de laboratorio (diamante), idéntica en los tres esquemas de encaminamiento. La línea roja marca el enlace con el cuello de botella configurado.

El Diagrama 1 es un esquema propio pensado para mostrar claramente el direccionamiento; la Captura 1 es la topología real tal como quedó armada en GNS3, con los cinco routers, el cliente y el servidor, y los tres enlaces de respaldo marcados en verde (enlace activo).



Captura 1. Topología de laboratorio en GNS3 con los cinco routers MikroTik CHR, el cliente y el servidor, y todos los enlaces activos (puntos verdes).

La Captura 2 muestra el mismo panel durante un ensayo de falla: el enlace primario R2–R4 (el del cuello de botella) queda en pausa – GNS3 lo marca con el ícono amarillo de enlace suspendido en lugar del punto verde –, mientras en el enlace de respaldo R5–R4 se ve el ícono de lupa que indica una captura de Wireshark activa, registrando el tráfico que efectivamente cruza por ese camino durante la ventana de indisponibilidad.



Captura 2. Ensayo de falla en GNS3: el enlace R2–R4 suspendido (ícono amarillo) y una captura de Wireshark activa sobre R5–R4 (ícono de lupa), registrando el tráfico de respaldo durante el corte.

La Tabla 16 vuelca el direccionamiento completo de cada enlace, extraído directamente de los exports de RouterOS (`/ip address` de cada router).

Enlace	Subred	Extremo A	Extremo B
Cliente – R1	10.10.1.0/24	Cliente: 10.10.1.10/24	R1 (`ether2`): 10.10.1.1/24
R1 – R2 (primario)	10.10.12.0/30	R1 (`ether3`): 10.10.12.1/30	R2 (`ether2`): 10.10.12.2/30
R1 – R3 (respaldo)	10.10.13.0/30	R1 (`ether4`): 10.10.13.1/30	R3 (`ether2`): 10.10.13.2/30
R2 – R4 (cuello)	10.10.24.0/30	R2 (`ether3`): 10.10.24.1/30	R4 (`ether2`): 10.10.24.2/30
R3 – R5 (respaldo)	10.10.35.0/30	R3 (`ether3`): 10.10.35.1/30	R5 (`ether2`): 10.10.35.2/30
R5 – R4 (respaldo)	10.10.45.0/30	R5 (`ether3`): 10.10.45.1/30	R4 (`ether3`): 10.10.45.2/30
R4 – Servidor	10.10.4.0/24	R4 (`ether4`): 10.10.4.1/24	Servidor: 10.10.4.10/24

Tabla 16. Direccionamiento IP por enlace (idéntico en estática, RIP y OSPF).

Anexo B – Configuraciones de routers

Cada router tiene su export de RouterOS (`export`, sin usuarios, contraseñas ni `system id`): 5 routers × 3 esquemas, 15 exports en total. El direccionamiento (Tabla 16) y la cola del cuello de botella son idénticos en los tres; lo que cambia es exclusivamente el bloque de ruteo. A continuación mostramos los fragmentos más relevantes de cada uno.

Cola y marcado del cuello de botella (igual en los tres esquemas)

Ejemplo del export de R2 en el esquema estático; R4 tiene la regla simétrica (`cuello-R4-R2` sobre `ether2`) para limitar también el sentido servidor-cliente.

```
/queue tree
add max-limit=500k name=cuello-R2-R4 packet-mark=cuello-R2-R4 parent=ether3
/ip firewall mangle
add action=mark-packet chain=forward comment=marca-cuello-R2-R4 \
    new-packet-mark=cuello-R2-R4 out-interface=ether3 passthrough=no
```

Rutas estáticas con monitoreo activo

Del export de R1 en el esquema estático. La ruta primaria usa `check-gateway=ping` sobre un monitoreo recursivo (no sobre R2 directamente, sino sobre la IP de R4 del otro lado del cuello); la de respaldo es una ruta flotante con distancia administrativa 2.

```
/ip route
add check-gateway=ping comment=primaria-R2 distance=1 dst-address=\
    10.10.4.0/24 gateway=10.10.24.2 target-scope=11
add comment=respaldo-R3-R5 distance=2 dst-address=10.10.4.0/24 gateway=\
    10.10.13.2
```

RIPv2

Del export de R1 en el esquema RIP. El mismo patrón se repite en R2 a R5, aplicando la instancia a las interfaces que corresponda en cada router.

```
/routing rip instance
add name=rip-v2 redistribute=connected
/routing rip interface-template
add instance=rip-v2 interfaces=ether3,ether4
add instance=rip-v2 interfaces=ether2
```

OSPF

Del export de R1 en el esquema OSPF. Los costos explícitos (10 para el primario, 20 para el respaldo) se repiten en R2 y R4 sobre sus interfaces correspondientes, tal como describimos en la Sección 2.2.

```
/routing ospf instance
add name=ospf-v2 redistribute=connected router-id=10.255.0.1
/routing ospf area
add area-id=0.0.0.0 instance=ospf-v2 name=backbone
/routing ospf interface-template
add area=backbone cost=10 interfaces=ether3
add area=backbone cost=20 interfaces=ether4
```

Anexo C – Capturas de Wireshark

Todas las capturas del experimento instrumentado se guardaron en formato `.pcap`. La Tabla 17 resume el inventario por esquema y ensayo; los archivos no se adjuntan a este documento por su tamaño. Cada prueba de carga conserva además su log de `ping`; las fallas agregan eventos, resultados HTTP y cinco logs de rutas activas.

Esquema	Ensayo	Archivos	Punto(s) de captura	Artefactos asociados
Estática	Carga: UDP, TCP, HTTP y slow-start	55	R2-R4	JSON/salida de carga y ping
Estática	Contención: TCP+UDP y HTTP+UDP	10	R2-R4	Resultados de ambos flujos y ping
Estática	Falla R2-R4	10	R2-R4 y R5-R4	Ping, HTTP, eventos y rutas R1–R5
RIP	Falla R2-R4	10	R2-R4 y R5-R4	Ping, HTTP, eventos y rutas R1–R5
OSPF	Falla R2-R4	10	R2-R4 y R5-R4	Ping, HTTP, eventos y rutas R1–R5

Tabla 17. Inventario de capturas PCAP del experimento instrumentado (95 archivos).

Las capturas de diagnóstico se conservan fuera de este inventario y no se usan para ningún valor del informe. En las corridas de falla, las capturas R2–R4 y R5–R4 se complementan con los logs de rutas de R1 a R5: en conjunto permiten verificar el corte del camino principal, la presencia de tráfico sobre el respaldo y el cambio de gateway que sustenta la métrica de convergencia de reenvío.

Anexo D – Registro de corridas

La Tabla 18 condensa las 80 corridas instrumentadas, agrupando las cinco repeticiones de cada condición en una sola fila. Todas conservan los parámetros literales, fecha, hora y artefactos crudos en la matriz; las pruebas de carga agregan RTT ICMP, las de arranque lento conservan `snd_cwnd` desde el inicio y las fallas registran rutas activas en R1–R5.

Esquema	Ensayo	Tráfico / parámetro	Repet.	Fecha (UTC)
Estática	Congestión	UDP 100 Kbit/s + RTT ICMP	r01–r05	2026-11-09
Estática	Congestión	UDP 200 Kbit/s + RTT ICMP	r01–r05	2026-11-09
Estática	Congestión	UDP 300 Kbit/s + RTT ICMP	r01–r05	2026-11-09
Estática	Congestión	UDP 400 Kbit/s + RTT ICMP	r01–r05	2026-11-09
Estática	Congestión	UDP 500 Kbit/s + RTT ICMP	r01–r05	2026-11-09
Estática	Congestión	UDP 600 Kbit/s + RTT ICMP	r01–r05	2026-11-09
Estática	Congestión	UDP 700 Kbit/s + RTT ICMP	r01–r05	2026-11-09
Estática	Congestión	UDP 800 Kbit/s + RTT ICMP	r01–r05	2026-11-09
Estática	Congestión	TCP referencia + RTT ICMP	r01–r05	2026-11-09
Estática	Congestión	HTTP/1.1 referencia + RTT ICMP	r01–r05	2026-11-09
Estática	Contención	TCP+UDP + RTT ICMP	r01–r05	2026-11-09
Estática	Contención	HTTP+UDP + RTT ICMP	r01–r05	2026-11-09
Estática	Arranque lento	TCP, intervalos 0,2 s y `snd_cwnd`	r01–r05	2026-11-09
Estática	Convergencia	HTTP/1.1+ICMP, falla R2-R4 y rutas R1–R5	r01–r05	2026-11-09
RIP	Convergencia	HTTP/1.1+ICMP, falla R2-R4 y rutas R1–R5	r01–r05	2026-11-09
OSPF	Convergencia	HTTP/1.1+ICMP, falla R2-R4 y rutas R1–R5	r01–r05	2026-11-09

Tabla 18. Registro condensado de corridas instrumentadas (16 series × 5 repeticiones = 80 corridas).